



INF1316

Sistemas Operacionais

LAB 1: CRIAÇÃO DE PROCESSOS HIERÁRQUICOS - fork()

Integrantes:

2320276 Aline Jéssica David Gonçalves

Professor(a):

Luiz Fernando Bessa Seibel

Rio de Janeiro

Agosto de 2026

1. OBJETIVO

O objetivo do programa é declarar uma variável inteira n com valor inicial = 1 e para criar 3 processos hierárquicos (pai, filho e neto). Após o `fork()` o processo pai faz um loop de 1000 vezes somando 1 à variável n , exibe a mensagem “processo pai, pid=xxxx, n=yyyy” e espera o filho terminar. O processo filho faz um loop de 1000 vezes somando 2 à variável n , exibe a mensagem “processo filho, pid=xxxx, n=yyyy” e espera o neto terminar. O processo neto faz um loop de 1000 vezes somando 3 à variável n , exibe a mensagem “processo neto, pid=xxxx, n=yyyy” e termina.

2. SAÍDA DO PROGRAMA

```
● @alinejess →/workspaces/INF1319/LAB1 (main) $ gcc main.c -o main
● @alinejess →/workspaces/INF1319/LAB1 (main) $ ./main
processo pai, pid=22640, n=1001
processo filho, pid=22641, n=2001
processo neto, pid=22642, n=3001
```

3. ESTRUTURA DO PROGRAMA E SOLUÇÃO

Bibliotecas utilizadas e definição de constante:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
```

Função principal:

```
int main(void) {
    int n = 1;
    int i;
    pid_t pid_filho, pid_neto;
    pid_filho = fork();
    if (pid_filho == 0) {
        pid_neto = fork();
        if (pid_neto == 0) {
            for (i = 0; i < 1000; i++) {
                n = n + 3;
            }
            printf("processo neto, pid=%d, n=%d\n", getpid(), n);
            exit(0);
        } else {
```

```
    for (i = 0; i < 1000; i++) {
        n = n + 2;
    }
    printf("processo filho, pid=%d, n=%d\n", getpid(), n);
    wait(NULL);
    exit(0);
}
} else {
    for (i = 0; i < 1000; i++) {
        n = n + 1;
    }
    printf("processo pai, pid=%d, n=%d\n", getpid(), n);
    wait(NULL);
}
return 0;
}
```

A lógica do programa é dividida em 3 partes, uma para cada processo (pai, filho e neto):

1ª parte: a criação do processo filho. No início da `main()`, a variável `n` é inicializada com valor 1 e, em seguida, é chamada a primeira função `fork()`, que pertence ao processo do filho. Nesse momento, o sistema operacional cria uma cópia completa do processo original, incluindo uma cópia independente da variável `n`. A partir daqui, existem dois processos em execução: o pai (processo original) e o filho (processo recém-criado).

2ª parte: Diferenciação do comportamento com o retorno do `fork()`. Como o `fork()` retorna 0 apenas para o processo filho, o `if (pid_filho == 0)` é usado para direcionar cada processo para o bloco de código correspondente à sua função no programa. Dentro do bloco do filho ocorre a segunda chamada de `fork()`, responsável por criar o neto. Essa segunda chamada é feita antes de o filho executar seu próprio laço de repetição, garantindo que o neto herde o mesmo valor de `n = 1` que o filho recebeu do pai, e não um valor já alterado.

3ª parte: Execução dos laços e sincronização com `wait()`. Cada processo (pai, filho e neto) executa um laço `for` de 1000 iterações, somando a `n` o valor definido no enunciado (1, 2 e 3) e depois imprime sua mensagem com `getpid()` e o valor final de `n`. O neto, por não ter nenhum processo descendente, apenas encerra sua execução com `exit(0)` após imprimir sua mensagem. Já o filho e o pai chamam `wait(NULL)` logo após a impressão, para respeitar a hierarquia de espera exigida pelo enunciado: o

filho aguarda o neto terminar, e o pai aguarda o filho terminar, antes de cada um encerrar sua própria execução.

Essa estrutura garante que os três processos existam simultaneamente e executem seus laços de forma independente e paralela, mas ao mesmo tempo respeitem uma ordem de encerramento hierárquica (neto termina primeiro, depois filho, depois pai).

4. OBSERVAÇÕES

Dificuldades encontradas:

A primeira vez que utilizamos o `fork()` na disciplina foi com um status, então foi preciso adaptar a função do `fork()` para executar sem o status e com 3 processos simultâneos.

Facilidades encontradas:

Ligado com o ponto descrito acima, foi mais simples entender e implementar a lógica de 3 processos finitos simultâneos alterando apenas a lógica do `pid` de cada um dos processos.

Compilação:

O programa foi compilado e testado no ambiente de desenvolvimento do GitHub de linguagem C.

5. CONCLUSÃO

O programa evidenciou, na prática, que a chamada `fork()` cria processos com espaços de memória completamente independentes, e não uma referência compartilhada. Por isso, mesmo com pai, filho e neto executando seus laços em paralelo sobre uma variável de mesmo nome (`n`), cada um manipula sua própria cópia dessa variável, sem interferência entre si. Isso explica os resultados obtidos (`n = 1001` no pai, `n = 2001` no filho e `n = 3001` no neto), que correspondem exatamente ao esperado, uma vez que todos partem do mesmo valor inicial (`n = 1`) e apenas aplicam seus próprios incrementos, e também justifica por que esses valores são reprodutíveis entre execuções, uma vez que a ausência de memória compartilhada elimina qualquer condição de corrida (race condition) sobre `n`, diferente do que ocorreria se os mesmos fluxos fossem implementados como threads.